

MLflow Tracking Lab

Experiment Tracking & Model Registry

0. Prerequisites

Before running this lab, ensure the following are installed and configured on your machine.

0.1 Python Environment

- Python 3.8 or higher
- A virtual environment is strongly recommended

```
python -m venv .venv
source .venv/bin/activate    # Mac/Linux
.venv\Scripts\activate      # Windows
```

0.2 Required Libraries

Install all dependencies with:

```
pip install pandas numpy scikit-learn mlflow matplotlib
```

Library	Purpose
<code>pandas</code>	Data loading and manipulation
<code>numpy</code>	Numerical computing
<code>scikit-learn</code>	ML algorithms, preprocessing, pipelines
<code>mlflow</code>	Experiment tracking and model registry
<code>matplotlib</code>	Plotting (confusion matrix figure)

0.3 Dataset

The lab uses a financial transactions dataset stored as a pickle file at `../data/data.pkl` relative to the notebook. The dataset contains labelled transactions (fraud vs. legitimate) with the following key columns:

Column	Description
<code>step</code>	Hour of the transaction (1–744)
<code>type</code>	Transaction type: CASH_OUT, PAYMENT, TRANSFER, CASH_IN, DEBIT

<code>amount</code>	Amount transferred
<code>nameOrig / nameDest</code>	Anonymised account IDs (C = customer, M = merchant)
<code>oldbalanceOrig / newbalanceOrig</code>	Sender balance before and after
<code>oldbalanceDest / newbalanceDest</code>	Receiver balance before and after
<code>isFraud</code>	Target label: 1 = fraudulent, 0 = legitimate
<code>isFlaggedFraud</code>	System rule-based flag (not ML-generated)

1. Exploratory Data Analysis (EDA)

The EDA section gives a quick overview of the dataset's structure and the distribution of the target variable. Three cells explore the data.

1.1 Transaction Type Distribution

The first EDA cell counts how many transactions belong to each type:

```
df.type.value_counts()

# Output:
# CASH_OUT      10840
# PAYMENT       6443
# TRANSFER      5655
# CASH_IN       4227
# DEBIT         111
```

CASH_OUT is the dominant type, followed by PAYMENT and TRANSFER. This distribution matters because, as we will see, fraud only occurs in specific types.

1.2 Target Class Balance

The second cell checks the proportion of fraudulent vs. legitimate transactions:

```
df.isFraud.value_counts(normalize=True)

# Output:
# 0      0.699  (legitimate)
# 1      0.301  (fraud)
```

💡 Key insight: the dataset is intentionally balanced (~30 % fraud rate) for this lab. In production, fraud is typically < 1 % of transactions, requiring special class-imbalance handling (oversampling, class_weight, etc.).

1.3 Fraud by Transaction Type

The third cell shows which transaction types are involved in fraud:

```
df[df.isFraud == 1].type.value_counts(normalize=True)
```

```
# Output:  
# CASH_OUT    0.501  
# TRANSFER    0.499
```

Fraud is exclusively associated with CASH_OUT and TRANSFER transactions. PAYMENT, CASH_IN, and DEBIT are never fraudulent in this dataset. This is a crucial feature-engineering insight: transaction type is a strong signal.

2. Data Preprocessing

2.1 The wrangle() Function

The wrangle() function applies all feature-engineering and cleaning steps in a single, reproducible pass. It operates on a copy of the DataFrame to avoid side-effects.

```
def wrangle(df):  
    df = df.copy()  
    cols = []  
  
    # 1. Drop feature-leaking columns  
    cols.append("newbalanceDest")  
    cols.append("newbalanceOrig")  
  
    # 2. Convert step (hour of simulation) → hour of day (1-24)  
    df["time"] = df["step"].apply(lambda step: (step - 1) % 24 + 1)  
    cols.append("step")  
  
    # 3. Drop the rule-based system flag  
    cols.append("isFlaggedFraud")  
  
    # 4. Reduce account IDs to their category prefix (C or M)  
    df["nameOrig"] = df["nameOrig"].str[0]  
    df["nameDest"] = df["nameDest"].str[0]  
  
    df.drop(columns=cols, inplace=True)  
    return df
```

2.2 What is Feature Leakage?

Feature leakage (also called data leakage) happens when a feature used for training contains information that would not be available at prediction time in a real deployment. Using such features produces artificially inflated metrics during evaluation, but the model fails in production.

⚠ A feature leaks if it is generated **AFTER** or **AS A DIRECT CONSEQUENCE** of the event you are predicting.

Example: `newbalanceDest` and `newbalanceOrig` are the account balances **AFTER** the transaction has completed — and after fraud has already occurred. At the moment a transaction is being screened, those updated balances do not yet exist. Using them would be like letting the model peek at the answer sheet.

Dropped Feature	Why it leaks
<code>newbalanceDest</code>	Balance of destination account AFTER the transaction — only exists once money moved.
<code>newbalanceOrig</code>	Balance of origin account AFTER the transaction — only exists once money moved.
<code>isFlaggedFraud</code>	Label produced by the rule-based system AFTER reviewing the transaction. Using it would leak the outcome.

2.3 Other Transformations

Transformation	Rationale
<code>step → time</code>	<code>step</code> encodes hours since simulation start (1–744). $(\text{step}-1) \% 24 + 1$ maps it to hour-of-day (1–24), a more meaningful cyclic feature.
<code>nameOrig</code> / <code>nameDest</code>	Account IDs like 'C123456' are too granular (near-unique). Keeping only the first character ('C' or 'M') encodes the account category without overfitting.

3. MLflow Setup

Setting up MLflow requires answering three fundamental questions before writing a single line of tracking code.

Question 1 — Where is the MLflow server?

MLflow needs to know where to send tracking data. This is configured with:

```
mlflow.set_tracking_uri(MLFLOW_URI)
```

URI Value	Behaviour
<code>""</code> (empty string)	Local filesystem. MLflow creates an <code>mlruns/</code> folder in the current working directory. Default and easiest option for local experiments.
<code>"http://host:5000"</code>	Remote MLflow server. Metadata is sent over HTTP to a shared server — useful for team collaboration.

"sqlite:///mlflow.db"	Local SQLite database. A step up from the filesystem; enables full SQL querying of experiment history.
-----------------------	--

 Local mode (empty string): When `MLFLOW_URI = ""`, running `set_tracking_uri("")` tells MLflow to store everything in `./mlruns/` relative to the notebook directory. A default experiment named 'Default' is created automatically. A WARNING is shown in recent versions recommending a database backend for production use.

Question 2 — Where do I save my metadata?

MLflow metadata is all the structured information recorded about a run. It is stored in the tracking **store (filesystem folder or database)**.

Metadata Type	Description
Parameters	Hyperparameters and configuration values (e.g., <code>C=1.0</code> , <code>max_iter=100</code>)
Metrics	Scalar evaluation results (e.g., <code>f1_score=0.92</code> , <code>accuracy=0.94</code>)
Tags	Free-form key-value labels (e.g., <code>team='data-science'</code> , <code>version='v1'</code>)
Run metadata	Start time, end time, status (FINISHED / FAILED), git commit hash
Artifact paths	Pointers to files stored in the artifact store (not the files themselves)

Question 3 — Where do I store artifacts?

Artifacts are the files produced by a run: **model binaries, plots, datasets, etc.** They are stored in the artifact store, which is configured separately from the tracking store.

Storage	Configuration
Local folder	Default when using filesystem tracking. Artifacts saved inside <code>./mlruns/<experiment>/<run>/artifacts/</code>
Amazon S3	<code>s3://my-bucket/mlruns</code> — recommended for production and team use
Azure Blob	<code>wasbs://container@account.blob.core.windows.net/path</code>
GCS	<code>gs://my-bucket/mlruns</code>

3.1 What is an MLflow Experiment?

An Experiment is the top-level organisational unit in MLflow. Think of it as a folder that groups all the runs associated with a given modelling task or project.

Concept	Meaning
Experiment	A named container for related runs (e.g., 'fraud_detection')
Run	A single execution of your training code, recorded inside an experiment
Artifact	File outputs of a run: model file, plot, dataset sample...
Parameter	An input to the run — recorded before or during training

Metric	An output of the run — a scalar value recorded at one or more steps
---------------	---

By default, MLflow uses an experiment called 'Default'. To create or switch to your own named experiment:

```
# This creates the experiment if it does not exist,
# or activates it if it already does.
mlflow.set_experiment("fraud_detection")

# Output (first run):
# INFO mlflow.tracking.fluent: Experiment 'fraud_detection' does not exist.
# Creating a new experiment.
```

💡 Best practice: always call `set_experiment()` explicitly. This ensures all your runs land in the right experiment regardless of the working directory or who runs the notebook.

4. Building the Model Pipeline

The lab uses a scikit-learn Pipeline to chain preprocessing and modelling into a single, clean object. This pattern prevents data leakage between train and test sets and makes the model easy to deploy.

4.1 Why a Pipeline?

- Applies the same preprocessing steps to training and test data automatically.
- Prevents test-set information from leaking into the preprocessor's fit (e.g., OneHotEncoder categories).
- Packages the entire model (preprocessor + classifier) as one serialisable object.
- Integrates cleanly with MLflow's sklearn model logging.

4.2 ColumnTransformer: handling mixed types

The dataset has both categorical columns (type, nameOrig, nameDest) and numerical columns. ColumnTransformer applies different transformations to different subsets of columns:

```
cat_cols = df.select_dtypes("object").columns.tolist()
# → ['type', 'nameOrig', 'nameDest']

preprocessor = ColumnTransformer(
    transformers=[
        ("ohe", OneHotEncoder(handle_unknown="ignore"), cat_cols)
    ],
    remainder="passthrough"  # numeric columns pass through unchanged
)
```

Argument	Effect
<code>ohe</code>	Applies OneHotEncoder to the categorical columns
<code>handle_unknown="ignore"</code>	Silently ignores categories seen in test but not in train
<code>remainder="passthrough"</code>	All other columns (numeric) are kept as-is

4.3 Full Pipeline

```
lr_pipe = make_pipeline(
    preprocessor,
    LogisticRegression()
)

# make_pipeline is equivalent to:
# Pipeline([("columntransformer", preprocessor), ("logisticregression",
# LogisticRegression())])
```

When `lr_pipe.fit(X_train, y_train)` is called, the pipeline: (1) fits the OneHotEncoder on `X_train`'s categorical columns, (2) transforms `X_train`, and (3) trains LogisticRegression on the resulting feature matrix. When `lr_pipe.predict(X_test)` is called, steps (1) and (2) are applied with the already-fitted encoder — no re-fitting on test data.

5. Tracking the Experiment

5.1 The with mlflow.start_run() Pattern

All tracking calls must happen inside a run context. The `with` statement ensures the run is properly closed (status set to `FINISHED`) even if an exception occurs:

```
with mlflow.start_run():
    # Everything logged here belongs to this run.
    # The run is automatically ended when the block exits.
    ...
```

💡 You can also name your run for easier identification in the UI:
`with mlflow.start_run(run_name="logistic_baseline"):`

5.2 TODO Solutions

Complete each TODO in the tracking cell using the hints and solutions below.

TODO 1 — Log the dataset

Logging the dataset links the exact data used to the run, enabling full reproducibility.

TODO

✓ Solution

Log dataset (hint: log_input)

```
dataset = mlflow.data.from_pandas(df,
source=DATAPATH, targets="isFraud" )
mlflow.log_input(dataset, context="training")
```

TODO 2 — Log metrics

Metrics are scalar numbers that measure model quality. Log them after evaluation:

TODO

Log metrics (hint: log_metrics)

✓ Solution

```
y_pred = lr_pipe.predict(X_test) f1 =
f1_score(y_test, y_pred) mlflow.log_metrics({
"f1_score": f1, })
```

TODO 3 — Log hyperparameters

Parameters describe the configuration of the model. They allow you to compare runs with different settings:

TODO

Log hyperparameters (hint: log_params)

✓ Solution

```
lr = lr_pipe.named_steps["logisticregression"]
mlflow.log_params({ "C": lr.C,
"max_iter": lr.max_iter, "solver":
lr.solver, "test_size": 0.2,
"random_state": RANDOM_STATE, })
```

TODO 4 — Log the model

Logging the model saves the entire pipeline (preprocessor + classifier) as a reusable MLflow artifact:

TODO

Log model (hint: mlflow.sklearn.log_model)

✓ Solution

```
mlflow.sklearn.log_model(sk_model=lr_pipe,
artifact_path="model",
input_example=X_train[:5], )
```

TODO 5 — Log the confusion matrix figure

Figures (matplotlib, plotly) can be logged as image artifacts alongside the model:

TODO

Log confusion matrix (hint: log_figure)

✓ Solution

```
fig, ax = plt.subplots(figsize=(6, 5)) cm =
confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(cm).plot(ax=ax)
ax.set_title("Confusion Matrix")
mlflow.log_figure(fig, "confusion_matrix.png")
plt.close(fig)
```

5.3 Complete Tracking Block

Putting it all together:

```
with mlflow.start_run(run_name="logistic_baseline"):
```

```
# Dataset
```



```

dataset = mlflow.data.from_pandas(df, source=DATAPATH, targets="isFraud")
mlflow.log_input(dataset, context="training")

# Train
lr_pipe.fit(X_train, y_train)
y_pred = lr_pipe.predict(X_test)

# Metrics
mlflow.log_metrics({"f1_score": f1_score(y_test, y_pred)})

# Hyperparameters
lr = lr_pipe.named_steps["logisticregression"]
mlflow.log_params({"C": lr.C, "max_iter": lr.max_iter, "solver": lr.solver})

# Model
mlflow.sklearn.log_model(lr_pipe, artifact_path="model")

# Confusion matrix
fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay(confusion_matrix(y_test, y_pred)).plot(ax=ax)
mlflow.log_figure(fig, "confusion_matrix.png")
plt.close(fig)

```

6. Model Registry

6.1 What is the Model Registry?

The MLflow Model Registry is a centralised catalogue that manages the lifecycle of ML models across teams and environments. It is separate from experiment tracking: while tracking records every run, the registry only holds models that have been explicitly promoted.

Concept	Description
Registered Model	A named model in the registry (e.g., 'fraud_detector'). Contains one or more versions.
Model Version	A specific logged model promoted to the registry. Automatically versioned (v1, v2, ...).
Stage	Lifecycle label: None → Staging → Production → Archived
Alias	Human-readable pointer to a version (e.g., 'champion', 'challenger')

Typical workflow: train → log model in a run → register best run in registry → promote to Staging for validation → promote to Production for serving.

6.2 When to Use the Registry

- When multiple models (or model versions) need to coexist.
- When you need to roll back to a previous version quickly.
- When a deployment pipeline fetches the 'Production' model by name.
- When teams need an approval process before a model goes live.

6.3 TODO — Register the Model

There are two approaches to registering a model.

Option A — Register during the run (recommended)

```
with mlflow.start_run():
    ...
    mlflow.sklearn.log_model(
        sk_model=lr_pipe,
        artifact_path="model",
        registered_model_name="fraud_detector",  # ← registers automatically
    )
```

Option B — Register after the run (from a run URI)

```
# After the run has ended, retrieve the run_id
run_id = mlflow.last_active_run().info.run_id

model_uri = f"runs://{run_id}/model"

mlflow.register_model(
    model_uri=model_uri,
    name="fraud_detector",
)
```

6.4 Promoting a Model Version

After registering, assign an alias to mark the version ready for production:

```
from mlflow import MlflowClient

client = MlflowClient()

# Assign a 'champion' alias to version 1
client.set_registered_model_alias(
    name="fraud_detector",
    alias="champion",
    version=1,
)
```

6.5 Loading a Registered Model

In a serving or scoring script, load the model directly from the registry without needing the original training code:

```
import mlflow.sklearn

# Load by alias
model = mlflow.sklearn.load_model("models:/fraud_detector@champion")

# Load by version number
model = mlflow.sklearn.load_model("models:/fraud_detector/1")

# Use it like any sklearn model
predictions = model.predict(X_new)
```

✅ Best practice: in production pipelines, always load by alias ('champion', 'production') rather than by version number. This decouples your serving code from specific version IDs: you can promote a new version simply by updating the alias — no code changes needed.

6.6 Viewing the Registry

Launch the MLflow UI to browse registered models, compare versions, and manage stages:

```
mlflow ui # opens http://127.0.0.1:5000
```

Navigate to the 'Models' tab in the left sidebar to see all registered models, their versions, and their current aliases.